

实验一 简单任务调度及优先队列算法

一、实验目的

1. 掌握任务调度的基本原理和实现方法。
2. 学习优先队列的使用及其在任务调度中的应用。
3. 实现基于 Rust 的任务调度和优先队列算法，学习 Rust 异步编程和并发机制。
4. 分析不同优先级的任务调度效果。

二、实验原理

2.1 任务调度

任务调度是计算机系统中一种用于管理和分配资源的机制。它的目标是优化系统性能、提高资源利用率以及确保多任务环境中的公平性和优先级。通过任务调度，系统能够协调多个任务或作业，决定它们的执行顺序、开始时间和执行资源。

2.2 常见的调度策略

先来先服务（FCFS）
短作业优先（SJF）
优先级调度
时间片轮转（RR）
多级反馈队列（MLFQ）

2.3 实验重点

优先队列是一种特殊的队列，元素按优先级排序，优先级高的元素先出队。在任务调度中，优先队列可用于动态调整任务的执行顺序。

本实验基于 Rust 编程语言实现：

- 一个带有优先级的任务调度器。
- 三种优先级队列（低、普通、高）。

三、实验内容

1. 实现一个任务结构，包含任务名称（或标识符）和优先级。
2. 实现优先队列，支持任务的进队和出队操作。
3. 通过例子进行测试，实现按优先级调度任务。

四、实验步骤

4.1 代码结构概览

- Task: 定义任务结构体, 包括任务 ID、异步任务和优先级。
- Executor: 任务执行器, 管理任务队列和任务调度。
- TaskWaker: 自定义 Waker, 用于唤醒任务。

4.2 核心代码说明

src/task/mod.rs

```
use alloc::boxed::Box;
use core::{
    future::Future,
    pin::Pin,
    sync::atomic::{AtomicU64, Ordering},
    task::{Context, Poll},
};

pub mod executor;
pub mod keyboard;
pub mod simple_executor;

/// 任务优先级枚举
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub enum Priority {
    Low = 0,    // 低优先级
    Normal = 1, // 普通优先级
    High = 2,   // 高优先级
}

/// 任务结构体
pub struct Task {
    id: TaskId,                // 任务 ID
    future: Pin<Box<dyn Future<Output = ()>>>, // 任务的 Future
    priority: Priority,        // 任务优先级
}

impl Task {
    /// 创建一个新的任务
    ///
    /// # 参数
    ///
    /// - `future`: 任务的 Future
```

```

    /// - `priority`: 任务优先级
    ///
    /// # 返回值
    ///
    /// 返回一个新的 Task 实例
    pub fn new(future: impl Future<Output = ()> + 'static, priority:
Priority) -> Task {
        Task {
            id: TaskId::new(),
            future: Box::pin(future),
            priority,
        }
    }

    /// 轮询任务以推进其状态
    ///
    /// # 参数
    ///
    /// - `context`: 任务的上下文
    ///
    /// # 返回值
    ///
    /// 返回任务的轮询状态
    fn poll(&mut self, context: &mut Context) -> Poll<()> {
        self.future.as_mut().poll(context)
    }
}

/// 任务 ID 结构体
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
struct TaskId(u64);

impl TaskId {
    /// 创建一个新的任务 ID
    ///
    /// # 返回值
    ///
    /// 返回一个新的 TaskId 实例
    fn new() -> Self {
        static NEXT_ID: AtomicU64 = AtomicU64::new(0);
        TaskId(NEXT_ID.fetch_add(1, Ordering::Relaxed))
    }
}

```

src/task/executor.rs

```
use super::{Task, TaskId};
use alloc::{collections::BTreeMap, sync::Arc, task::Wake};
use core::task::{Context, Poll, Waker};
use crossbeam_queue::ArrayQueue;

/// Executor 负责管理和调度任务。
pub struct Executor {
    /// 存储任务 ID 与对应任务的映射。
    tasks: BTreeMap<TaskId, Task>,
    /// 不同优先级的任务队列，使用数组存储。
    task_queues: [Arc<ArrayQueue<TaskId>>; 3],
    /// 缓存每个任务的 Waker，避免重复创建。
    waker_cache: BTreeMap<TaskId, Waker>,
}

impl Executor {
    /// 创建一个新的 Executor 实例，并初始化任务队列和 Waker 缓存。
    pub fn new() -> Self {
        Executor {
            tasks: BTreeMap::new(),
            task_queues: [
                Arc::new(ArrayQueue::new(100)), // 低优先级队列
                Arc::new(ArrayQueue::new(100)), // 普通优先级队列
                Arc::new(ArrayQueue::new(100)), // 高优先级队列
            ],
            waker_cache: BTreeMap::new(),
        }
    }

    /// 启动一个新任务，将其插入到对应优先级的队列中。
    ///
    /// # 异常
    ///
    /// 如果任务 ID 已存在或队列已满，会触发恐慌。
    pub fn spawn(&mut self, task: Task) {
        let task_id = task.id;
        let priority = task.priority as usize;
        // 将任务插入到 tasks 映射中，如果任务 ID 已存在则触发恐慌
        if self.tasks.insert(task.id, task).is_some() {
            panic!("task with same ID already in tasks");
        }
        // 将任务 ID 推入对应优先级的任务队列中
        self.task_queues[priority]
```

```

        .push(task_id)
        .expect("queue full");
    }

    /// 运行所有准备好的任务，通过轮询它们并管理其状态。
    fn run_ready_tasks(&mut self) {
        // 按照优先级从高到低遍历任务队列
        for priority in (0..3).rev() {
            // 不断从当前优先级的队列中弹出任务 ID
            while let Some(task_id) =
self.task_queues[priority].pop() {
                // 获取对应的任务，如果任务不存在则跳过
                let task = match self.tasks.get_mut(&task_id) {
                    Some(task) => task,
                    None => continue,
                };

                // 提前获取任务的优先级和对应的队列
                let task_priority = task.priority;
                let task_queue = self.task_queues[task_priority as
usize].clone();

                // 获取或创建该任务的 Waker
                let waker = self
                    .waker_cache
                    .entry(task_id)
                    .or_insert_with(|| TaskWaker::new(task_id,
task_queue));

                // 创建一个 Context 对象用于轮询任务
                let mut context = Context::from_waker(waker);
                // 轮询任务以推进其状态
                match task.poll(&mut context) {
                    Poll::Ready(()) => {
                        // 如果任务完成，移除任务和对应的 Waker
                        self.tasks.remove(&task_id);
                        self.waker_cache.remove(&task_id);
                    }
                    Poll::Pending => {
                        // 如果任务未完成，保留在队列中等待下次轮询
                    }
                }
            }
        }
    }
}

```

```

    }

    /// 如果没有准备好的任务，则将 CPU 置于低功耗状态。
    pub fn sleep_if_idle(&self) {
        use x86_64::instructions::hlt;
        hlt(); // 使 CPU 进入暂停状态，直到下一个中断
    }

    /// 持续运行准备好的任务，并在空闲时休眠。
    ///
    /// 此函数不会返回。
    pub fn run(&mut self) -> ! {
        loop {
            self.run_ready_tasks(); // 执行所有准备好的任务
            self.sleep_if_idle(); // 如果没有任务，休眠以节省资源
        }
    }
}

/// TaskWaker 负责通过重新将任务 ID 推入任务队列来唤醒任务。
struct TaskWaker {
    /// 需要唤醒的任务 ID。
    task_id: TaskId,
    /// 共享的任务队列，用于重新调度任务。
    task_queue: Arc<ArrayQueue<TaskId>>,
}

impl TaskWaker {
    /// 创建一个新的 Waker 实例，用于特定的任务。
    ///
    /// # 参数
    ///
    /// - `task_id`：需要被唤醒的任务 ID。
    /// - `task_queue`：任务将被推入的任务队列。
    ///
    /// # 返回值
    ///
    /// 一个 `Waker` 实例，可以用于唤醒任务。
    fn new(task_id: TaskId, task_queue: Arc<ArrayQueue<TaskId>>) ->
Waker {
        Waker::from(Arc::new(TaskWaker {
            task_id,
            task_queue,
        }))
    }
}

```

```

    }

    /// 将任务 ID 推回到任务队列中，以便下一轮调度。
    fn wake_task(&self) {
        self.task_queue.push(self.task_id).expect("task_queue
full");
    }
}

impl Wake for TaskWaker {
    /// 唤醒任务，将其 ID 推入任务队列中。
    ///
    /// # 参数
    ///
    /// - `self`: 一个指向 TaskWaker 实例的 Arc 引用。
    fn wake(self: Arc<Self>) {
        self.wake_task();
    }

    /// 通过引用唤醒任务，允许多个 Waker 实例引用同一个任务。
    ///
    /// # 参数
    ///
    /// - `self`: 一个指向 TaskWaker 实例的 Arc 引用。
    fn wake_by_ref(self: &Arc<Self>) {
        self.wake_task();
    }
}
}

```

Task 定义了任务的基本结构，包括任务的唯一 ID(TaskId)、执行逻辑(future)和优先级。

Executor 是调度器的核心，维护多个优先队列并按优先级执行任务。

4.3 实验测试方法

任务创建与调度测试：

- 创建多个任务，分别设置优先级为 High、Normal 和 Low。
- 调用 Executor::run 方法，验证高优先级任务是否优先执行。
- 在任务中模拟异步阻塞，使用 TaskWaker 唤醒任务，验证任务是否被正确放回队列并继续执行。

```

// 一个简单的异步任务
async fn example_task(id: &str) {
    println!("Task {} start", id);
    for i in 0..3 {

```

```

        println!("Task {} step {}", id, i);
    }
    println!("Task {} down", id);
}

```

```

fn kernel_main(boot_info: &'static BootInfo) -> ! {
    use blog_os::allocator;
    use blog_os::memory::{self, BootInfoFrameAllocator};
    use x86_64::VirtAddr;

    println!("Hello World{}", "!");
    blog_os::init();

    let phys_mem_offset =
VirtAddr::new(boot_info.physical_memory_offset);
    let mut mapper = unsafe { memory::init(phys_mem_offset) };
    let mut frame_allocator = unsafe
{ BootInfoFrameAllocator::init(&boot_info.memory_map) };

    allocator::init_heap(&mut mapper, &mut
frame_allocator).expect("heap initialization failed");

    #[cfg(test)]
    test_main();

    // 创建执行器
    let mut executor = Executor::new();

    // 创建不同优先级的任务
    executor.spawn(Task::new(
        example_task("Task1: High Priority"),
        Priority::High,
    ));
    executor.spawn(Task::new(
        example_task("Task2: Normal Priority"),
        Priority::Normal,
    ));
    executor.spawn(Task::new(
        example_task("Task3: Low Priority "),
        Priority::Low,
    ));

    executor.spawn(Task::new(
        example_task("Task4: Normal Priority "),

```



```

        Priority::Normal,
    ));

    executor.spawn(Task::new(
        example_task("Task5: High Priority "),
        Priority::High,
    ));

    // 启动键盘中断处理任务
    //      executor.spawn(Task::new(keyboard::print_keypresses(),
Priority::Low));

    // 运行执行器
    executor.run();
}

```

4.4 运行结果

```

Machine View
Task Task1: Highh Priority step 1
Task Task1: Highh Priority step 2
Task Task1: Highh Priority down
Task Task5: High Priority start
Task Task5: High Priority step 0
Task Task5: High Priority step 1
Task Task5: High Priority step 2
Task Task5: High Priority down
Task Task2: Normal Priority start
Task Task2: Normal Priority step 0
Task Task2: Normal Priority step 1
Task Task2: Normal Priority step 2
Task Task2: Normal Priority down
Task Task4: Normal Priority start
Task Task4: Normal Priority step 0
Task Task4: Normal Priority step 1
Task Task4: Normal Priority step 2
Task Task4: Normal Priority down
Task Task3: Low Priority start
Task Task3: Low Priority step 0
Task Task3: Low Priority step 1
Task Task3: Low Priority step 2
Task Task3: Low Priority down
WARNING: scancode queue uninitialized

```

可以看到虽然加入任务时的优先级散乱，但是执行时任务按照从高到低的优先级执行。证明调度算法发挥作用。

五、与现有实验相比的创新点

5.1 完善性

与传统调度算法相比，本实验加入了优先队列支持，能够动态调整任务优先级。使用 Rust 的 `async/await` 特性实现异步任务调度，符合现代编程趋势。通过本实验，可以深入理解任务调度的原理，掌握优先队列的应用，并提升对 Rust 异步编程模型的理解和实践能力。

5.2 创新点

设计了多优先级队列和基于优先级的动态调度策略。

自定义 **Waker** 实现了高效的任务唤醒机制。

提供了灵活的任务管理接口，支持扩展和多任务并发处理。